

GraphRAG × NCCN

Injected knowledge graphs, cited answers, glowing
flowcharts — now grounded in the patient record

NxtCure Labs

graphrag — uv workspace · Python 3.11 · Phoenix LiveView

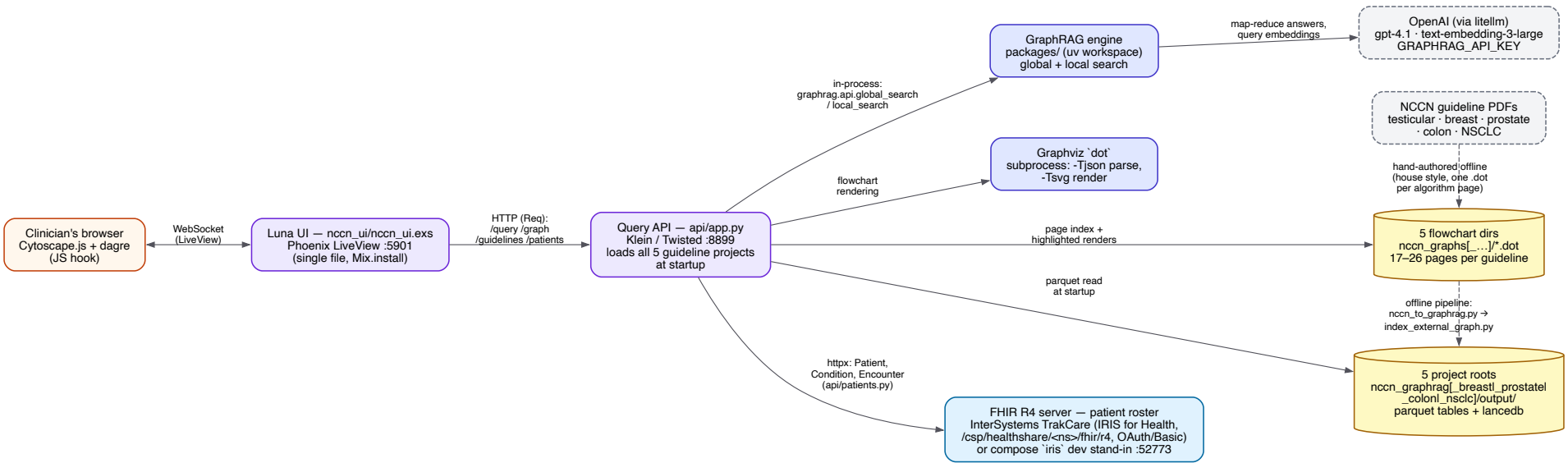
2026-09-11

Outline

- System context 2
- The engine underneath 5
- Injection 8
- Indexing 11
- Query API 14
- Evidence & rendering 17
- The Luna UI 20
- TrakCare integration 24
- Treatment-plan inference 30
- Facts & caveats 34

System context

- The core inversion: the knowledge graph is not LLM-extracted from text — it is **hand-authored** as Graphviz flowcharts, one .dot per NCCN algorithm page, and injected downstream of extraction.
- Five guidelines run side by side, each a full GraphRAG project: parquet tables, Leiden communities, LLM-written community reports, a LanceDB vector store.
- Four processes at runtime: the **LiveView UI** (:5901), the **Klein query API** (:8899, loads all five projects at startup), OpenAI behind the engine, and a **FHIR R4 server** for the patient roster — InterSystems TrakCare in production, an IRIS for Health container in development.
- Graphviz dot is a runtime dependency — the API shells out to it for parsing (-Tjson) and rendering (-Tsvg).

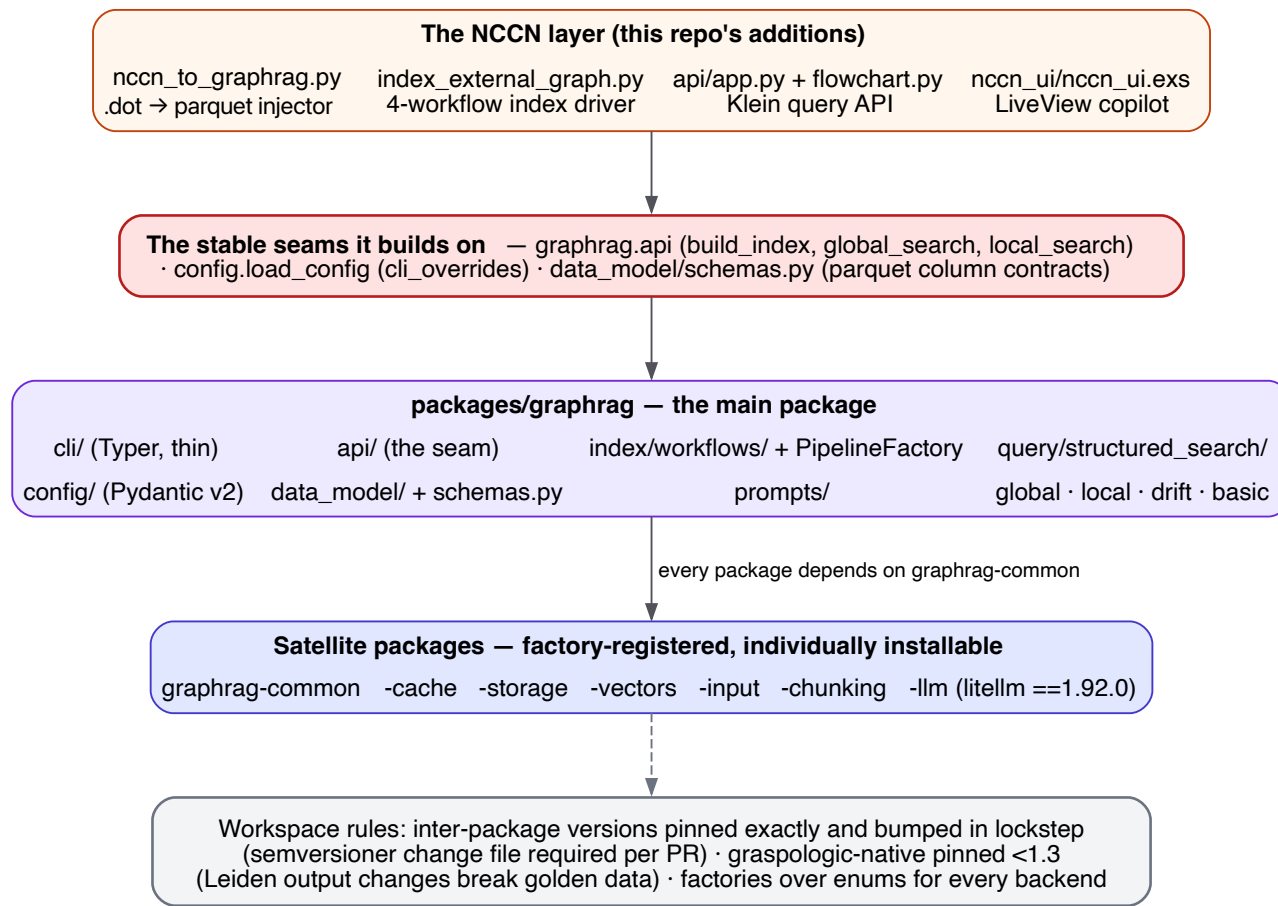


The engine underneath

A uv workspace, used only at its seams

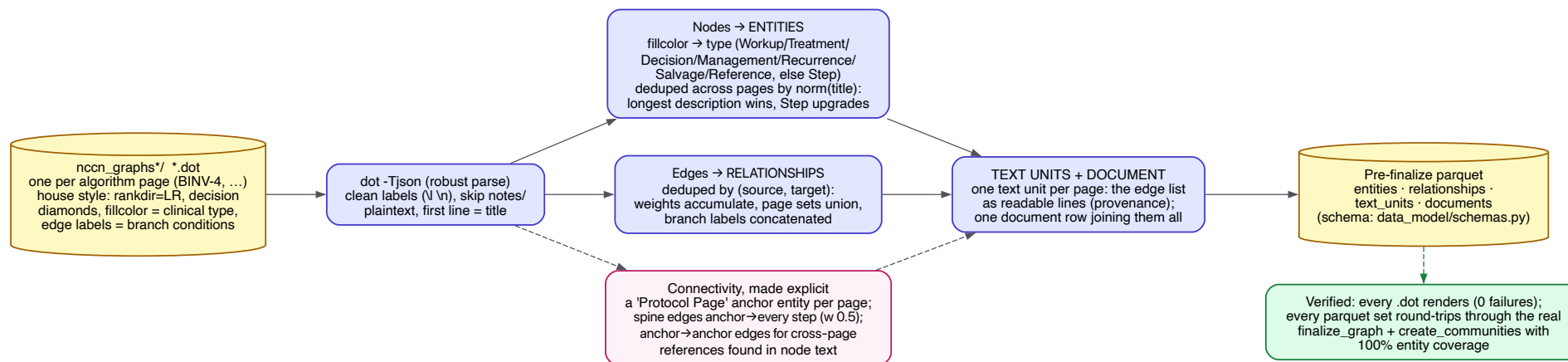
The engine underneath

- Upstream GraphRAG is a **uv workspace monorepo**: packages/graphrag (CLI, API, index + query engines, config, data model, prompts) plus seven factory-registered satellites, versions pinned in lockstep.
- The NCCN layer touches only the **stable seams**: `graphrag.api`, `load_config` with `cli_overrides`, and the `parquet` contracts in `schemas.py`.
- Nothing upstream is forked or patched — the whole implementation is four small additions: injector, index driver, query API, UI.
- Two pins matter: `graspologic-native <1.3` (Leiden output stability) and `litellm ==1.92.0` (all LLM traffic).



Injection

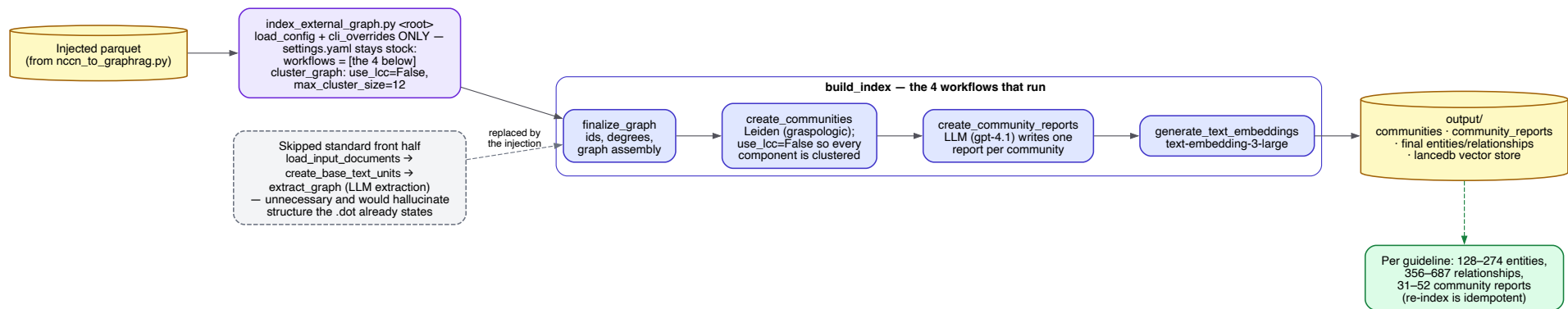
- Every flowchart **node** becomes an entity (fill color → clinical type) and every **edge** a relationship (branch labels kept as descriptions) — parsed via dot -Tjson.
- Concepts recurring across pages are **deduplicated** by normalized title: page sets merge, the longest description wins, weights accumulate.
- Connectivity is explicit: a “Protocol Page” **anchor entity** per page, spine edges to every step, and anchor→anchor edges for cross-page references.
- Provenance survives: one text unit per page — this is what later lets a citation point back to a specific flowchart.



Indexing

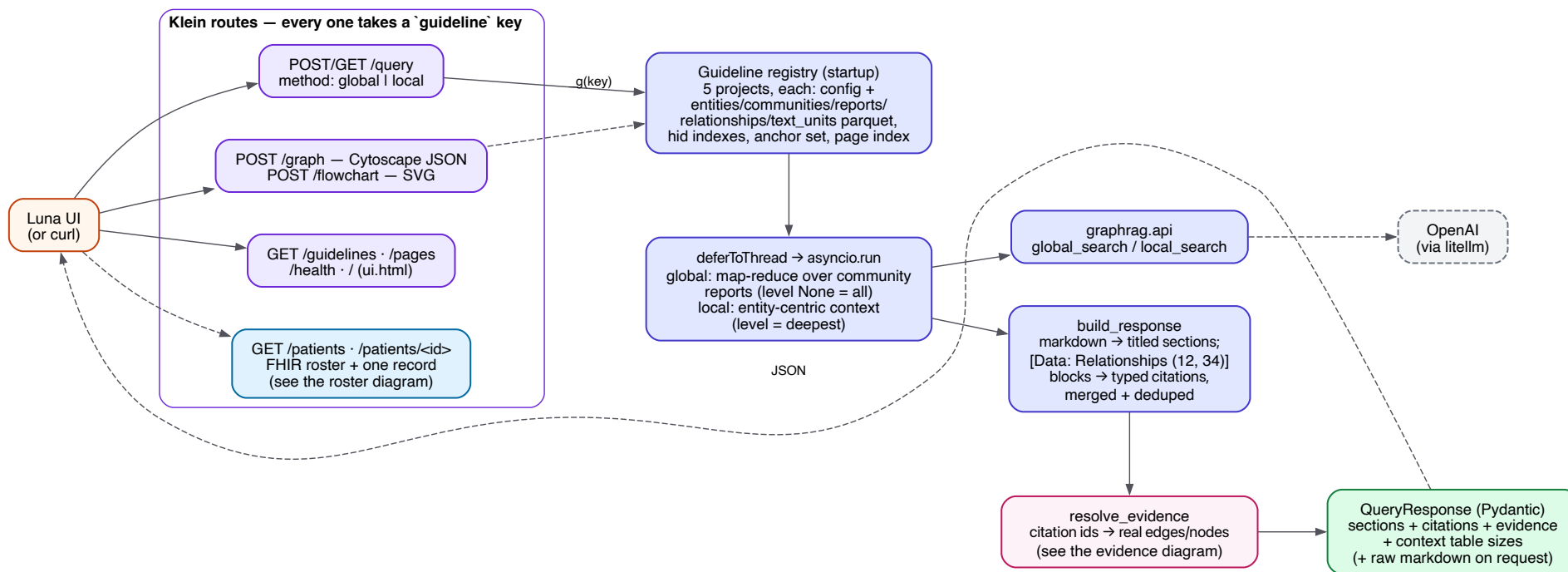
Half a pipeline, on purpose

- `index_external_graph.py` overrides **only the workflow list** — `settings.yaml` stays stock: finalize graph → communities → reports → embeddings.
- The standard front half (loading, chunking, LLM graph extraction) is skipped: the graph is already stated by the `.dot` files; extraction would only hallucinate on top.
- `use_lcc=False` matters: the injected graph has several components, and clustering only the largest would silently drop whole protocol areas.
- All sets round-trip the real pipeline with **100% entity coverage**; re-indexing is idempotent. The one LLM step: a report per Leiden community.



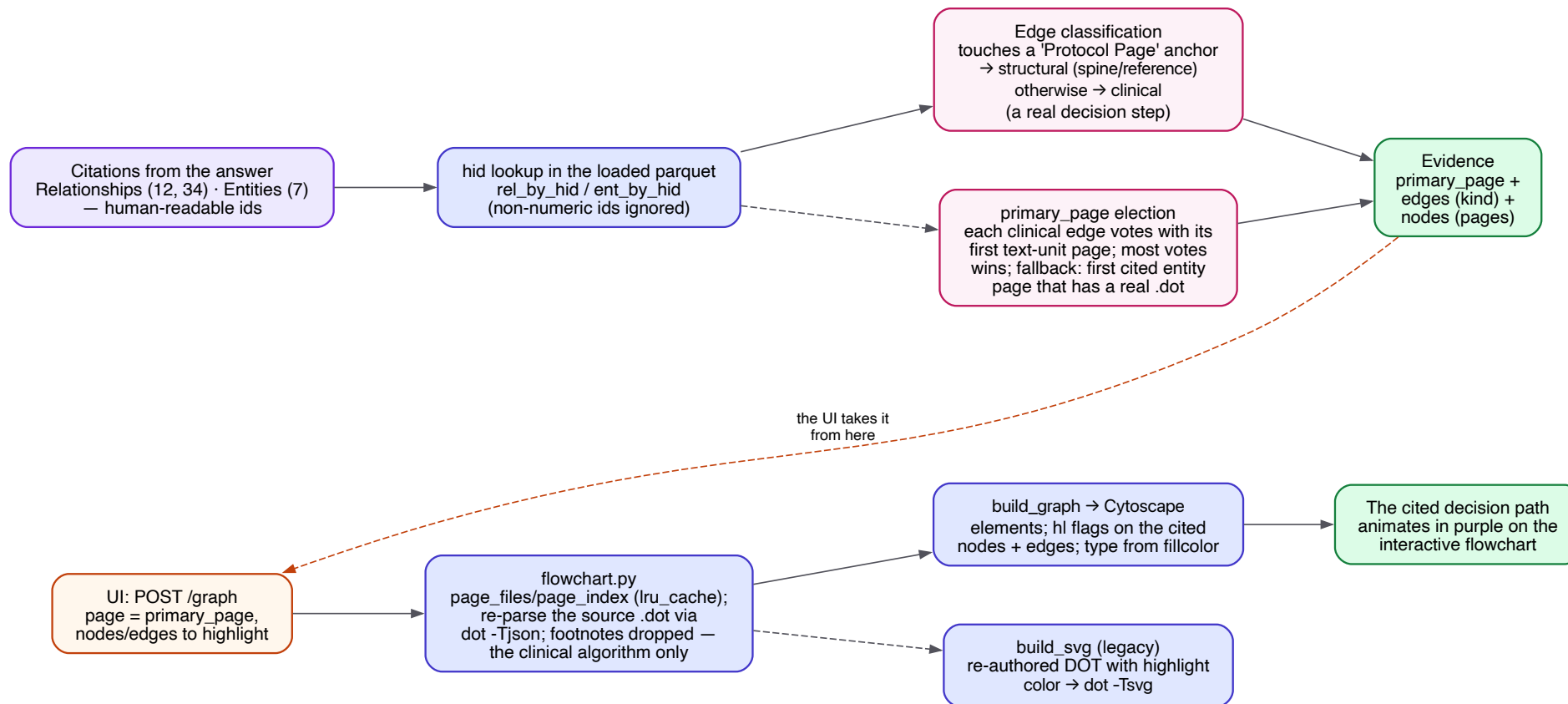
Query API

- One Klein/Twisted process serves all five guidelines; every endpoint takes a guideline key and dispatches to that project's config, tables, and flowcharts.
- Two search methods bridge to `graphrag.api` in a worker thread: **global** (map-reduce over community reports — thematic) and **local** (entity-centric — specific).
- Answer markdown becomes titled sections; [Data: Relationships (12, 34)] blocks become **typed citations**, merged and deduplicated.
- Citations are **resolved into evidence** server-side — the UI never has to understand GraphRAG's citation format.



Evidence & rendering

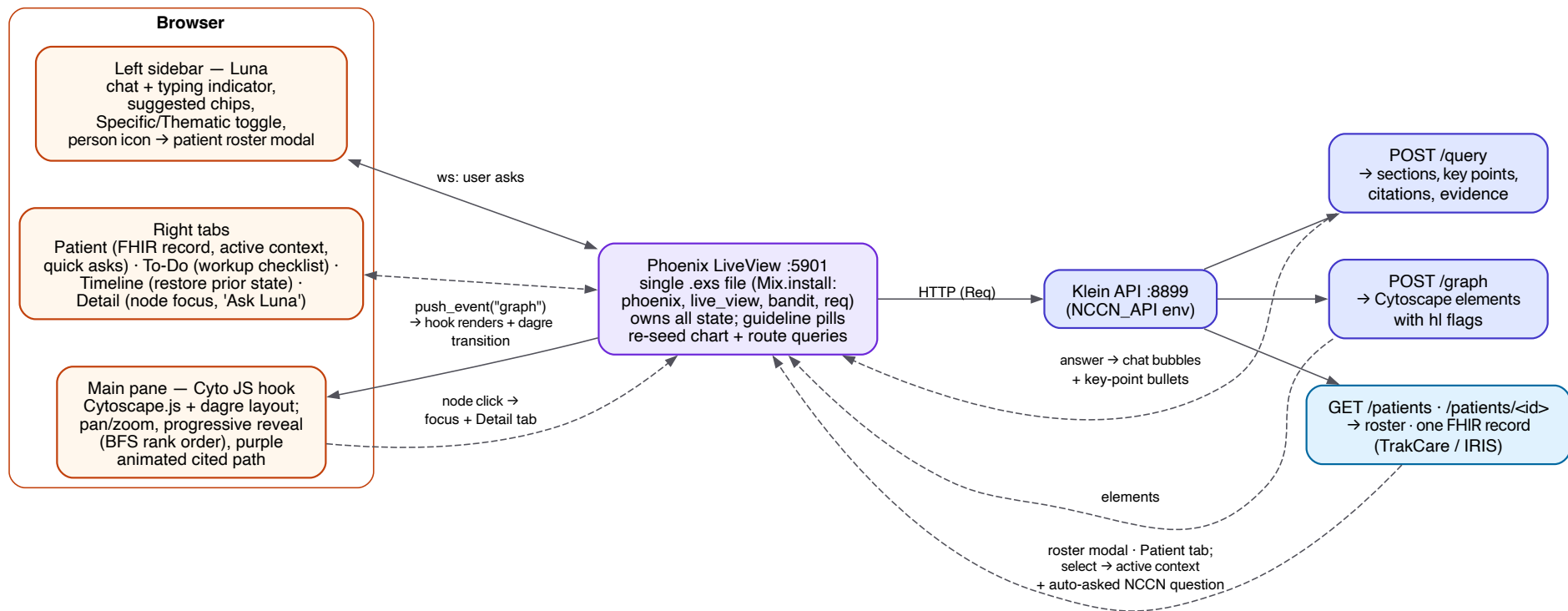
- Cited relationship ids are looked up and **classified**: edges touching a “Protocol Page” anchor are structural (bookkeeping); the rest are clinical — worth highlighting.
- Each clinical edge votes with its source page; most votes wins `primary_page` — the flowchart the UI should show for this answer.
- `flowchart.py` re-parses the **source** .dot (cached), drops footnotes, and emits Cytoscape elements with hl flags — or a re-rendered SVG (legacy).
- The result: the cited decision path animates in purple on the actual protocol page the answer came from.



The Luna UI

- A **single-file** Phoenix LiveView app (`Mix.install`, no scaffold) — the whole clinical copilot is one `.exs` file talking to the Python backend over HTTP.
- Ask a question → chat bubbles + key points land, evidence triggers a `/graph fetch`, and `push_event` hands the elements to the Cytoscape hook (dagre layout).
- The flowchart is explorable: progressive step reveal (rank-by-rank), decision diamonds, node click → Detail tab with “Ask Luna about this node”.
- To-Do (grounded workup checklist), Timeline (restore a prior state), and guideline pills (re-seed chart, reroute queries) complete the loop.

- A person icon beside Luna opens the **patient roster** — the bridge to the EHR covered in the next two sections.



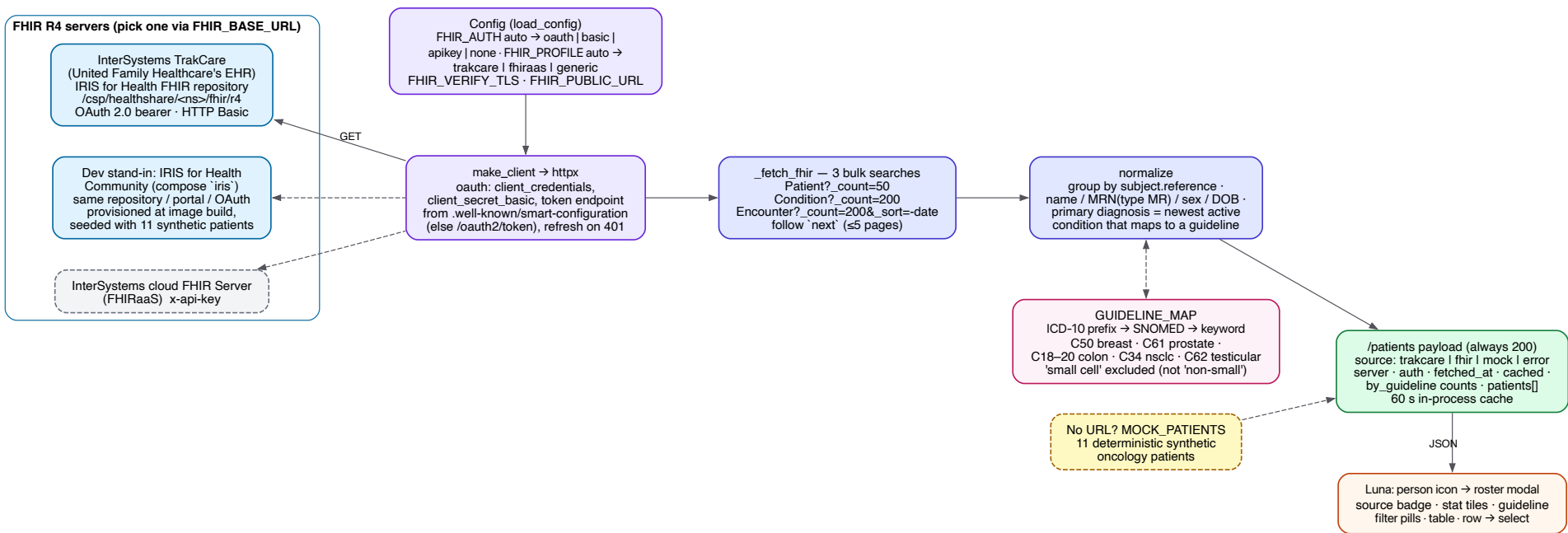
TrakCare integration

The EHR is the source of patients

TrakCare integration

- Beijing United Family Hospital — and the whole United Family Healthcare network — runs **InterSystems TrakCare**. Its FHIR layer is the IRIS for Health FHIR repository the TrakCare Innovation Toolkit installs: `https://<host>/csp/healthshare/<ns>/fhir/r4`, secured by the IRIS OAuth 2.0 server or HTTP Basic.
- `api/patients.py` is one **auth-aware FHIR R4 client**:
FHIR_AUTH=auto picks OAuth client_credentials (secret as client_secret_basic, token endpoint discovered from .well-known/smart-configuration, refreshed once on 401), HTTP Basic, or the cloud FHIR Server's x-api-key.

- Three **bulk searches** (Patient, Condition, Encounter) grouped client-side — not one round-trip per patient — then GUIDELINE_MAP turns each primary cancer diagnosis into an NCCN guideline key: ICD-10 prefix → SNOMED → keyword, with small-cell lung deliberately kept out of nsc1c.
- GET /patients always answers 200: source is trakcare, fhir, mock, or error, so the modal renders a friendly state instead of a stack trace. No URL configured → a deterministic synthetic roster.



No public sandbox — so we ship one

TrakCare integration

- TrakCare has **no self-service developer portal**; InterSystems grants sandbox access case by case. The compose stack therefore includes `iris: IRIS for Health Community`, provisioned **at image build** (`fhir-dev/iris.script`) with the same FHIR R4 endpoint shape, a Basic-auth user, and the Management Portal — then seeded once with 11 synthetic oncology patients (`fhir-seed`).
- Swapping to the real EHR is configuration only: `FHIR_BASE_URL + credentials + FHIR_PROFILE=auto` in `.env`. `FHIR_PUBLIC_URL` keeps the “Open FHIR Patient” links browser-reachable when the API talks to the server by another name.

No public sandbox — so we ship one

TrakCare integration

- `scripts/trakcare_fhir_setup.py` is a **human-in-the-loop Playwright walkthrough** of the IRIS Management Portal: it opens the right pages (Web Applications → find the endpoint, OAuth 2.0 → Client Descriptions, Users) and pauses with terminal instructions wherever a person must act; `--dev-iris` drives the dev container unattended.
- `make_oncology_bundle.py` emits a FHIR **transaction Bundle** mirroring the mock roster, so live and synthetic paths show identical data — and a round-trip through the normalizer is the regression test for the guideline mapping.

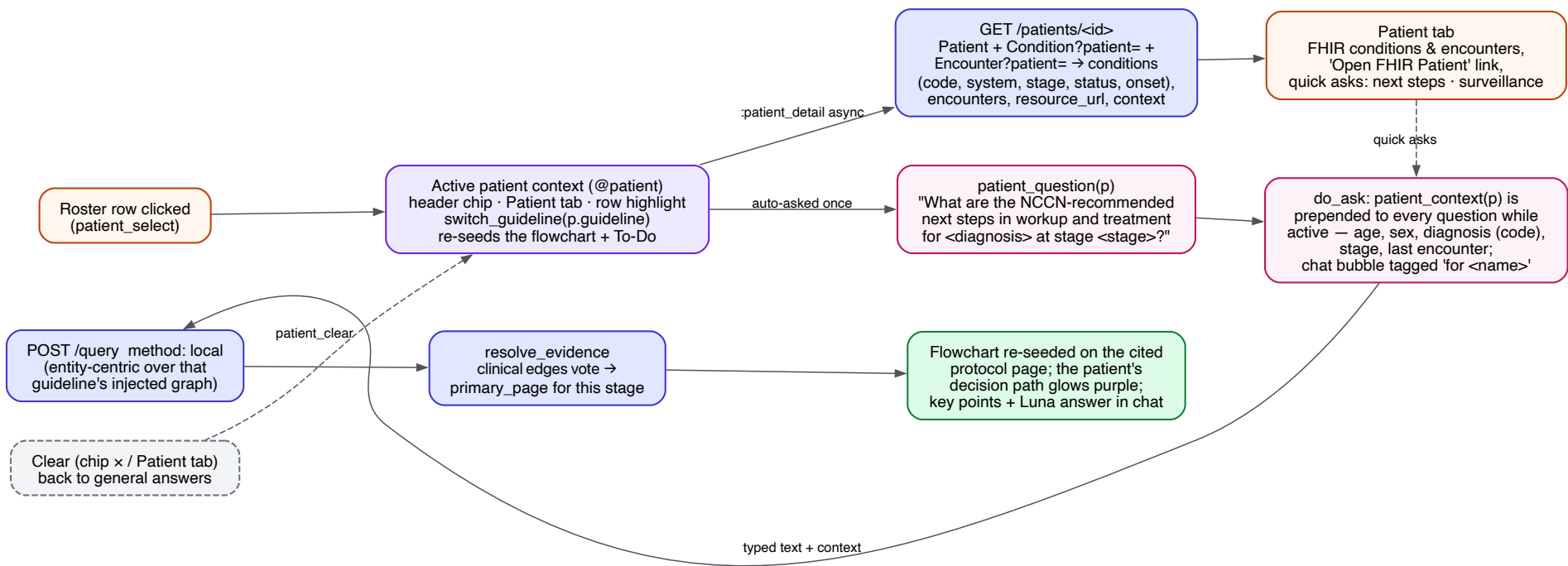
Treatment-plan inference

From a FHIR record to a glowing pathway

- Clicking a roster row makes that person the **active patient**: Luna switches to their guideline, a header chip and a Patient tab appear, and GET /patients/<id> pulls the full record — every Condition (code, system, stage, status, onset), recent Encounters, and a link to the raw FHIR resource.
- The inference step is a **grounded question**, not a new model: patient_question(p) asks what NCCN recommends next for that diagnosis at that stage, and patient_context(p) (age, sex, diagnosis + code, stage, last encounter) is prepended to **every** question while the patient is active. The chat shows what was typed, tagged “for <name>”.

From a FHIR record to a glowing pathway

- The existing machinery does the rest: local search over that guideline's injected graph → citations → clinical-edge votes → primary_page → the patient's decision path animates on the protocol page for their stage.
- Guardrails: context is built only from structured roster fields (no free-text PHI), the roster is read-only against the EHR, and the answer stays a **navigational aid** citing NCCN pages — the clinician reads the guideline, not the model.



Facts & caveats

Guideline	Flowcharts	Entities	Relationships	Reports
Breast (v5.2026)	24	217	534	51
Prostate (v2.2026)	18	143	487	33
Colon (v2.2026)	17	128	356	31
NSCLC (v6.2026)	26	274	687	52

- Coverage is the **primary pathways** of each guideline — representative, not every sub-page of the ~250-page documents; undrawn pages appear as dashed cross-page references.
- The flowcharts are lossy derivatives of copyrighted NCCN content: local use only, not redistribution, and never a substitute for the guideline itself.

- The roster is synthetic until a TrakCare host is configured: 11 patients (3 breast · 2 prostate · 2 colon · 2 NSCLC · 2 testicular), obviously fake names, deterministic. Patients whose conditions map to no guideline show as **Unmapped** and never drive Luna.

Start here

- `nccn_multicancer_README.md` — the whole story in one page
- `nccn_to_graphrag.py` — the injector, heavily commented
- `api/app.py` — registry, dispatch, evidence
- `api/patients.py` — the FHIR client, `GUIDELINE_MAP`, patient detail

The deep ends

- `packages/graphrag/graphrag/index/workflows/factory.py` — pipelines
- `packages/graphrag/graphrag/query/structured_search/` — the four search methods
- `nccn_ui/nccn_ui.exs` — the entire UI in one file (`patient_select`, `do_ask`)

- `fhir-dev/iris.script` ·
`scripts/`
`trakcare_fhir_setup.py` — the
dev EHR and the portal
walkthrough